

QUALITY ASSURANCE DIVISION

Enterprise Resource Allocation & Skill Management System (ERASM)

PROJECT DOCUMENTATION

Overview, Database Design, API Reference & Testing Evidence

Prepared By:	Raghav Bhat
Designation:	Information Science Engineering Student (Trainee)
Institution:	Vidyavardhaka College of Engineering (VVCE), Mysuru
Training Program:	Incture Technologies Training Program
Department:	Information Science and Engineering
Document Type:	Combined Project Report
Document Version:	1.0
Date of Issue:	July 5, 2026

Contents

1. Project Overview
2. Problem Statement
3. Database Design
4. ER Diagram
5. API List
6. Screenshots
7. Test Cases

1. Project Overview

The Enterprise Resource Allocation & Skill Management System (ERASM) is a backend enterprise application built to help organizations manage employees, projects, and the allocation of human resources across active initiatives. It is developed using Java 21 and Spring Boot 3, and exposes a secured set of RESTful APIs that handle authentication, employee records, project data, and percentage-based resource allocation, backed by a MySQL relational database through Hibernate/JPA.

The system is designed around ten functional modules covering user management, skill management, employee skill profiles, project management, resource request management, an approval workflow, resource allocation, a utilization dashboard, audit management, and reporting. It supports multiple user roles — Administrator, Manager, and Employee — each with distinct permissions enforced through JWT-based authentication and Spring Security's role-based access control.

Core capabilities include:

- Secure registration and login with JWT issuance and role claims (ROLE_ADMIN, ROLE_MANAGER, ROLE_EMPLOYEE).
- Full CRUD management of employees, departments, and projects.
- Skill and certification tracking against individual employee profiles.
- Resource allocation of employees to projects with a 100% cumulative allocation ceiling.
- Approval workflow for resource requests moving through Draft, Submitted, Review, Approved, Allocated, and Completed states.
- Audit logging of key actions (created/modified by, created/modified date) for accountability.
- Attendance, leave, and payroll tracking linked to each employee.

The application was validated through a structured software testing cycle combining JUnit 5/Mockito unit testing at the service layer and Postman-driven functional/API testing across the exposed REST endpoints, described in the sections that follow.

2. Problem Statement

Organizations running multiple concurrent projects often struggle to track which employees are available, which are over-allocated, and which skills are in short supply across the bench. Resourcing decisions are frequently made through spreadsheets or ad-hoc communication between Delivery Managers, Resource Managers, and project leads, which leads to inconsistent records, duplicate allocation of the same employee beyond 100% capacity, and a lack of auditable history for who approved a given resource request.

ERASM addresses this by centralizing employee, project, skill, and allocation data behind a single secured backend. It aims to:

- Provide a single source of truth for employee availability, skills, and certifications.
- Enforce allocation limits programmatically so an employee cannot be over-committed across concurrent projects.
- Formalize the resource-request-to-allocation workflow with explicit status transitions and guardrails against skipped states.
- Give managers visibility into utilization (Billable % and Bench %) derived from logged hours.
- Maintain an audit trail of who created or modified critical records and when.
- Restrict sensitive operations by role, so only authorized users can approve requests, allocate resources, or manage master data such as skills and departments.

The system is scoped strictly to backend services; frontend, mobile, and third-party cloud integrations are out of scope for this phase, as is large-scale performance/load testing beyond basic response-time observation.

3. Database Design

ERASM persists its data in MySQL 8.0 through Hibernate/Spring Data JPA. The schema is organized around an employees table as the central entity, linked outward to authentication (users, roles), organizational structure (departments), skill tracking (skills, employee_skills, certifications), project delivery (projects, resource_requests, allocations), workforce operations (attendance, leaves, payrolls, performance_reviews), and accountability (audit_logs).

3.1 Core Tables

users	Stores login credentials and account state: user_id (PK), username, email, password, enabled, and role_id (FK → roles).
roles	Master list of system roles: role_id (PK), role_name (e.g. ROLE_ADMIN, ROLE_MANAGER, ROLE_EMPLOYEE).
employees	Central employee record: employee_id (PK), user_id (FK → users), department_id (FK → departments), experience, designation, date_of_joining, salary, status.
departments	Organizational units: department_id (PK), department_name, description.
skills	Master skill catalog: skill_id (PK), skill_name, description.
employee_skills	Join table resolving the many-to-many relationship between employees and skills: composite key (employee_id, skill_id), plus experience_level and acquired_date.
certifications	Employee certifications: cert_id (PK), employee_id (FK → employees), certification_name, issued_by, issue_date, expiry_date.
projects	Project master data: project_id (PK), project_name, client_name, description, start_date, end_date, status, budget.
resource_requests	Requests for headcount against a project: request_id (PK), project_id (FK → projects), requested_by, request_date, required_skills, status.
allocations	Employee-to-project assignments: allocation_id (PK), employee_id (FK → employees), project_id (FK → projects), allocation_date, allocation_end_date, allocation_percentage.
performance_reviews	Review records: review_id (PK), employee_id (FK → employees), review_date, rating, comments.
attendance	Daily attendance: attendance_id (PK), employee_id (FK → employees), attendance_date, check_in_time, check_out_time, status, remarks.
leaves	Leave applications: leave_id (PK), employee_id (FK → employees), leave_type, start_date, end_date, status, reason.
payrolls	Payroll records: payroll_id (PK), employee_id (FK → employees), basic_salary, allowances, deductions, net_salary, payment_date, payroll_month.
audit_logs	System-wide audit trail: audit_id (PK), action_time, action, performed_by.

3.2 Key Relationships

- roles (1) — (N) users, via users.role_id.
- users (1) — (1) employees, via employees.user_id — each employee account maps to exactly one login.
- departments (1) — (N) employees, via employees.department_id.
- employees (M) — (N) skills, resolved through the employee_skills junction table.
- employees (1) — (N) certifications, allocations, attendance, leaves, payrolls, and performance_reviews.
- projects (1) — (N) allocations and resource_requests.
- audit_logs records actions performed across the system for traceability, independent of a single owning entity.

This design keeps employee identity (users/employees) separate from workforce operations (attendance, leave, payroll) and project delivery (projects, resource_requests, allocations), while employee_skills allows an employee to hold any number of skills at independent experience levels — supporting the skill-based resourcing that drives Module 5 (Resource Request Management) and Module 7 (Resource Allocation).

4. ER Diagram

The diagram below shows the full ERASM schema as implemented, including primary keys (yellow), foreign keys (red), and standard columns (blue), across all fifteen core tables.

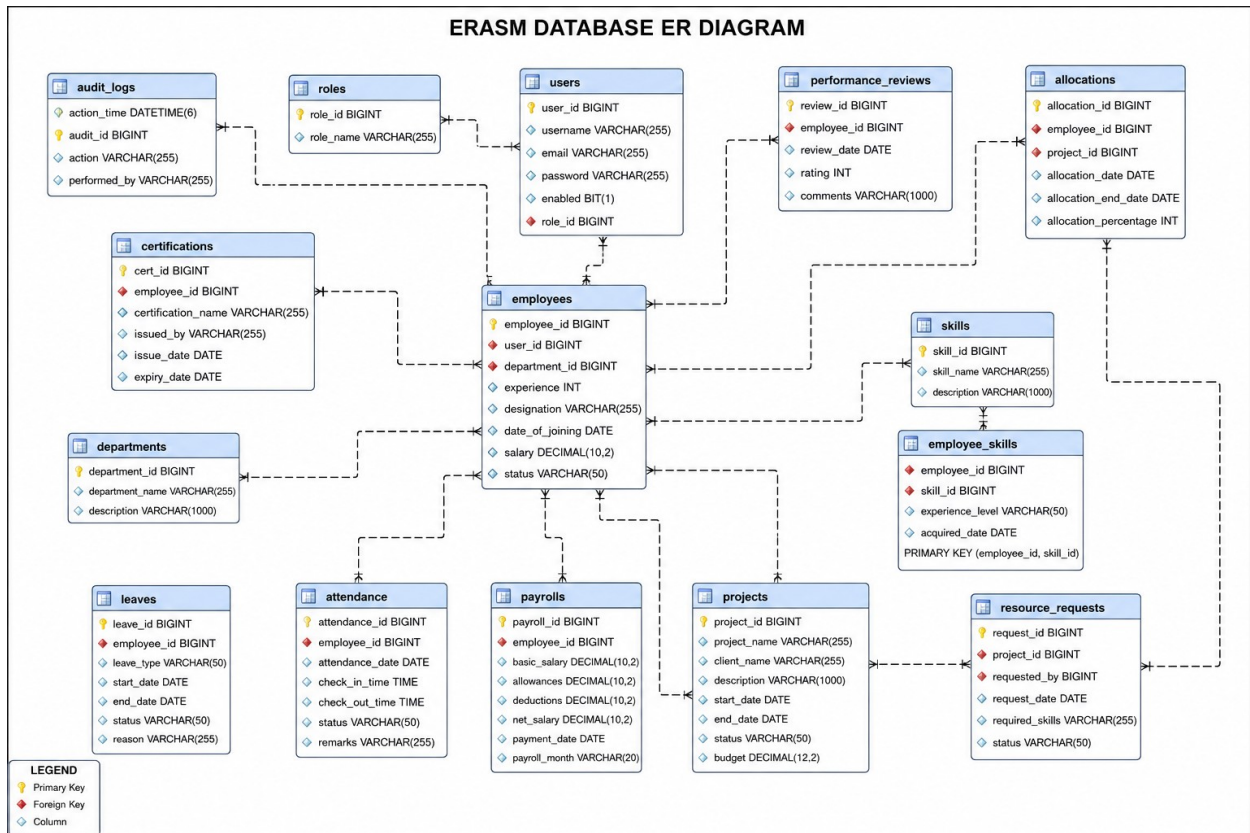


Figure A: ERASM Database ER Diagram

5. API List

The following REST endpoints were implemented and verified during the testing cycle described in Section 7 (Test Cases). All endpoints other than /auth/register and /auth/login require a valid JWT Bearer token, and write-restricted endpoints are additionally governed by @PreAuthorize role checks.

5.1 Authentication Module

Method	Endpoint	Description
POST	/auth/register	Register a new account with username, email, password, and roleId (1 = Admin, 2 = Employee, 3 = Manager).
POST	/auth/login	Authenticate with username/password and receive a signed JWT carrying the account's role claim.
POST	/auth/logout	Invalidate the client-side session for the presented Bearer token.

5.2 Employee Management Module

Method	Endpoint	Description
POST	/api/employees	Create an employee record (firstName, lastName, phone, departmentId, experience, userId).
GET	/api/employees	List all employees (Bearer-token protected).
GET	/api/employees/{id}	Retrieve a single employee by ID.
PUT	/api/employees/{id}	Update an employee's details, e.g. phone and department.
DELETE	/api/employees/{id}	Delete an employee record.

5.3 Project Management Module

Method	Endpoint	Description
GET	/api/projects	List all projects, including client name, technology stack, status, and budget.
PUT	/api/projects/{id}	Update a project's name, description, dates, status, and budget.
DELETE	/api/projects/{id}	Delete a project record.

5.4 Resource Allocation Module

Method	Endpoint	Description
POST	/api/allocations	Allocate an employee to a project at a given allocation percentage.
GET	/api/allocations/{id}	Retrieve an allocation with nested employee and department

Method	Endpoint	Description
		detail.
PUT	/api/allocations/release/{id}	Release an employee from an allocation and populate the release date.

Note: the ER diagram in Section 4 includes additional tables — skills, employee_skills, certifications, resource_requests, performance_reviews, attendance, leaves, and payrolls — that back Modules 2, 3, 5, 6, 8, 9, and 10 of the specification. Endpoints for these modules exist per the project's functional scope but were not exercised with Postman evidence in this testing cycle; if you have the corresponding controller routes or a Postman/OpenAPI export, they can be added to this list.

6. Screenshots

The screenshots below capture Postman requests and responses exercised during API testing, organized by module.

6.1 Authentication Module

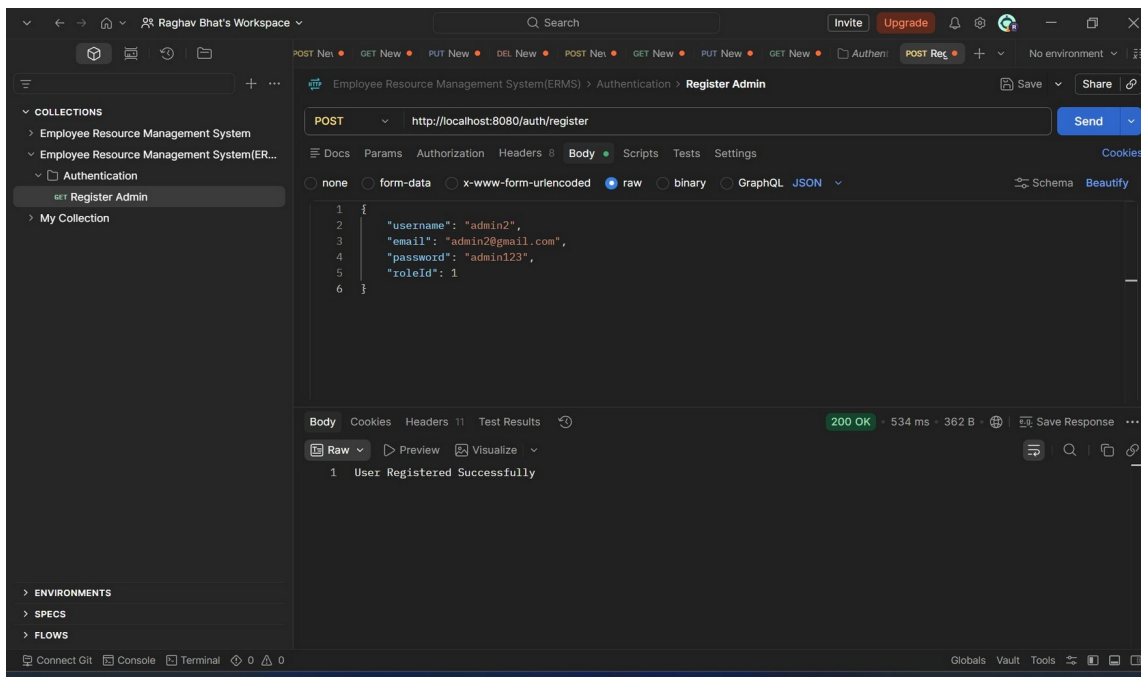


Figure 6: POST /auth/register — Admin account registration (roleId: 1)

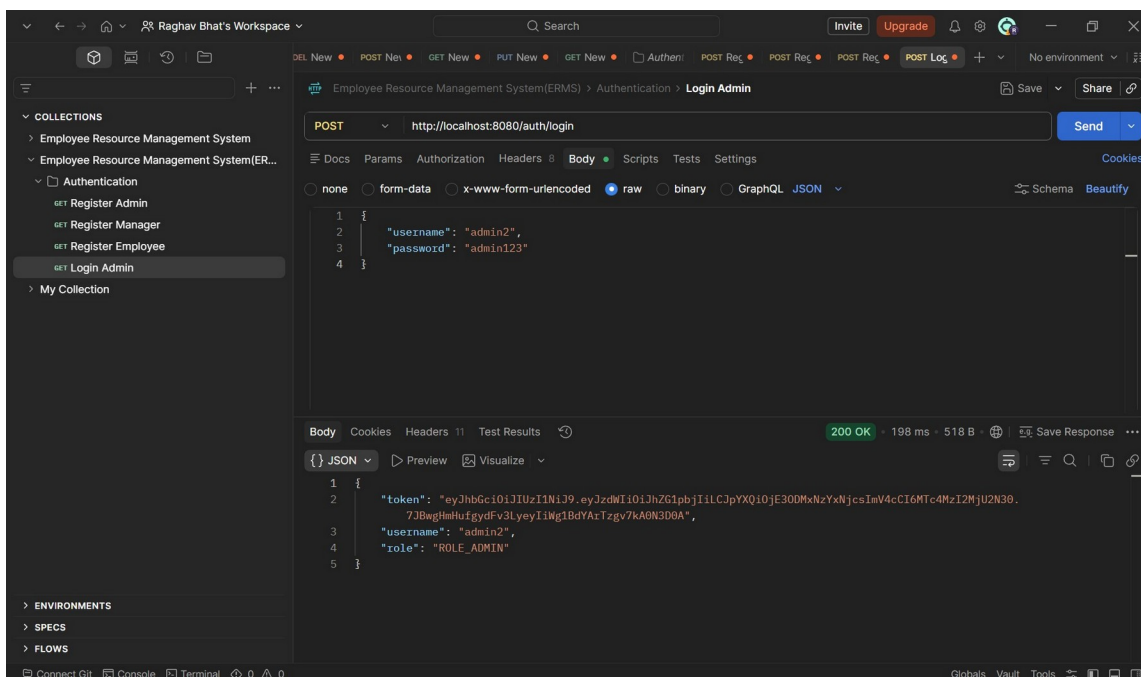


Figure 7: POST /auth/login — Admin login returning JWT with role ROLE_ADMIN

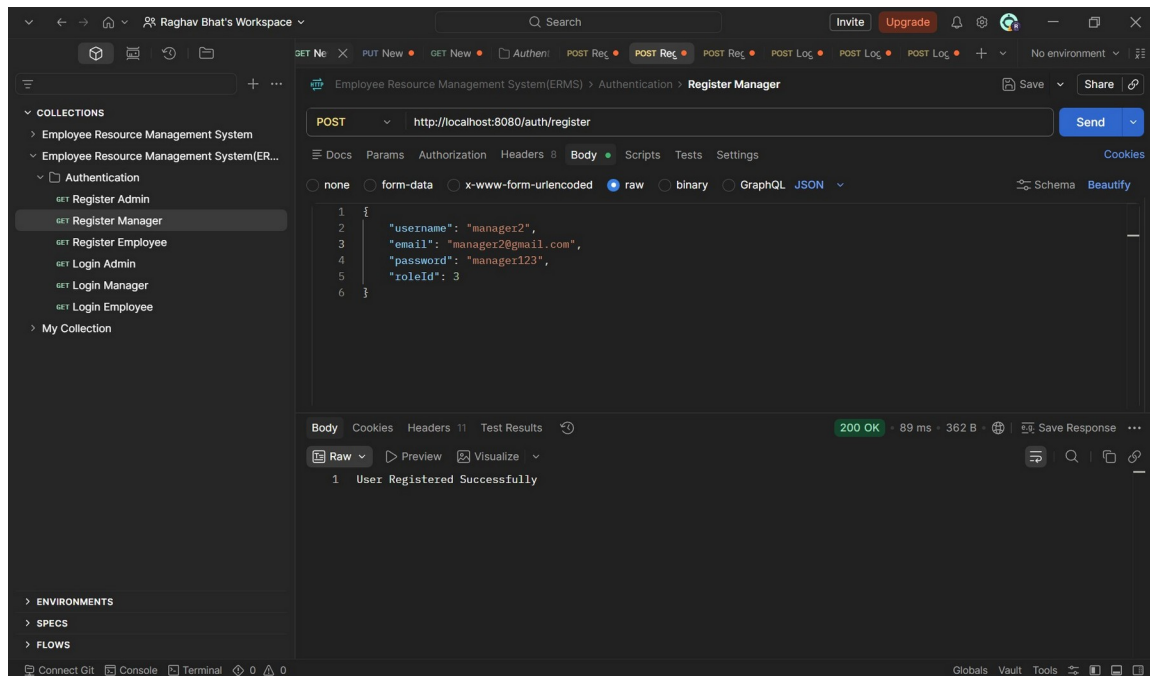


Figure 8: POST /auth/register — Manager account registration (roleId: 3)

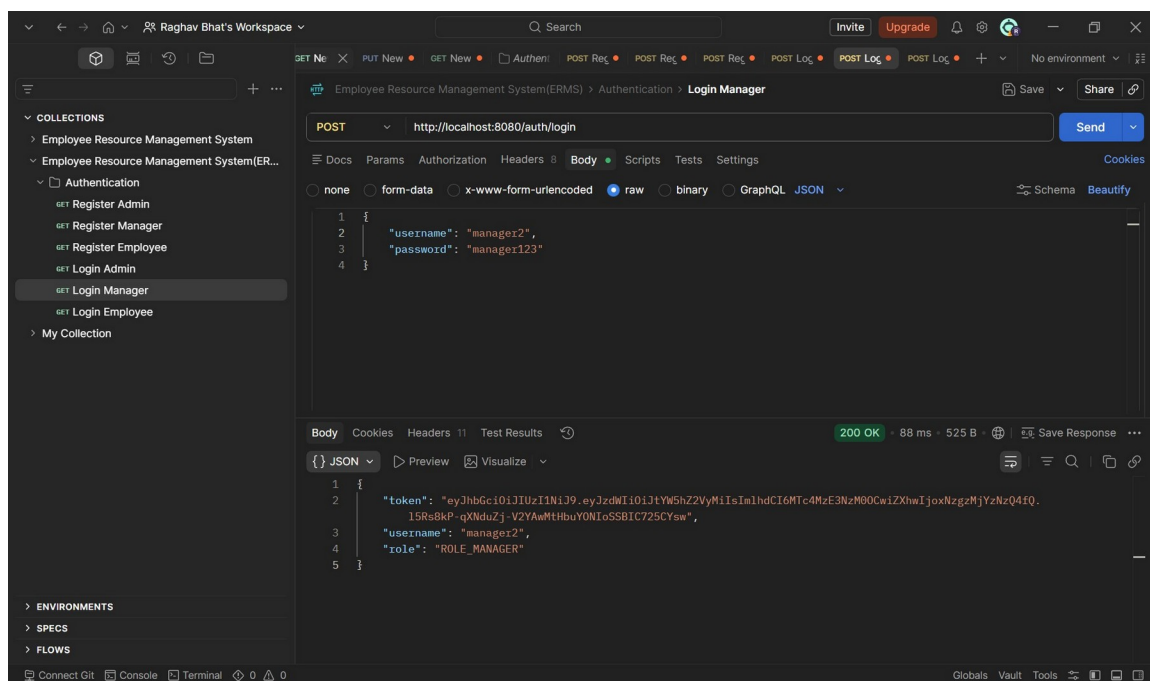


Figure 9: POST /auth/login — Manager login returning JWT with role `ROLE_MANAGER`

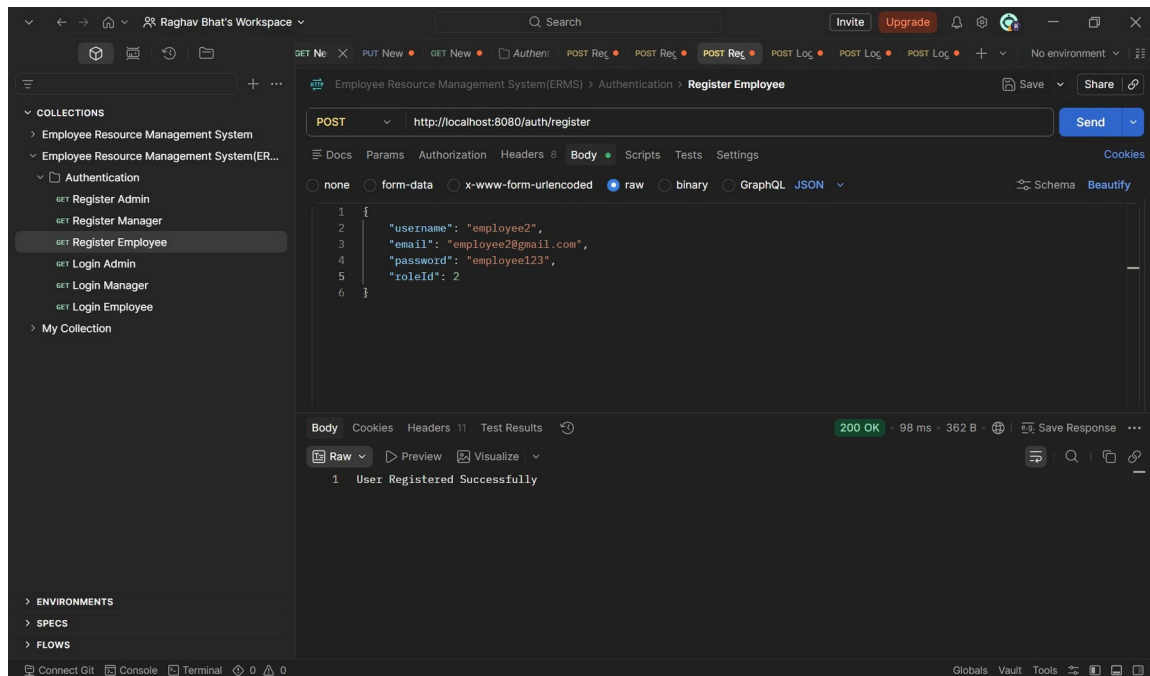


Figure 10: POST /auth/register — Employee account registration (roleId: 2)

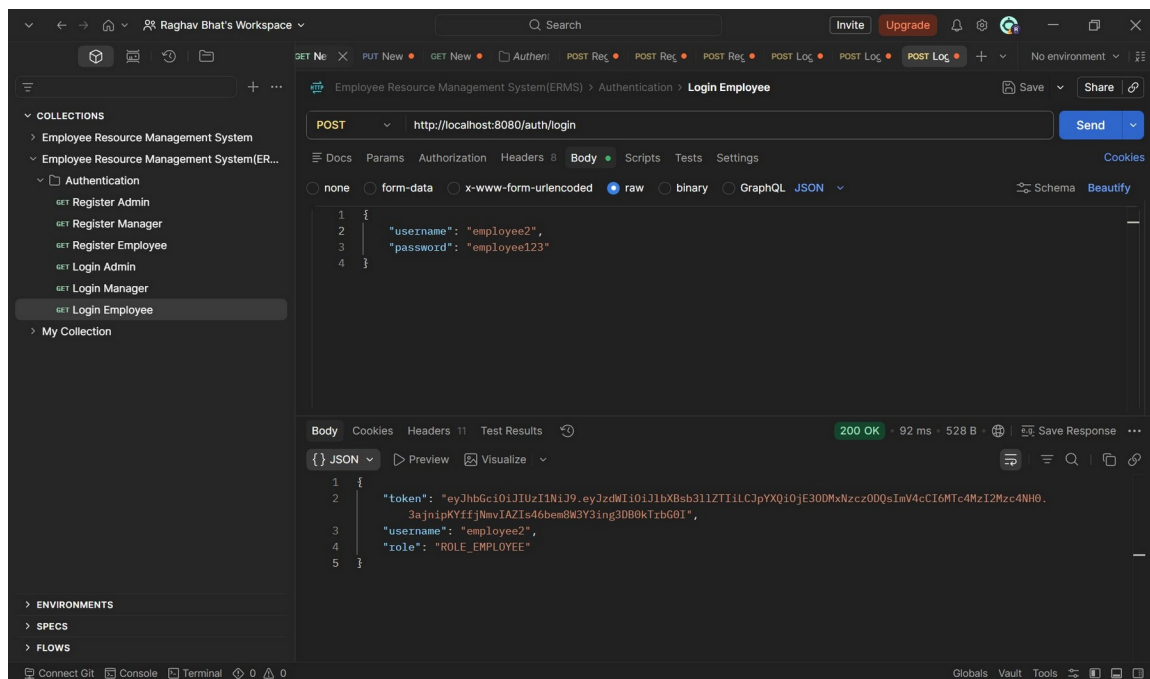


Figure 11: POST /auth/login — Employee login returning JWT with role ROLE_EMPLOYEE

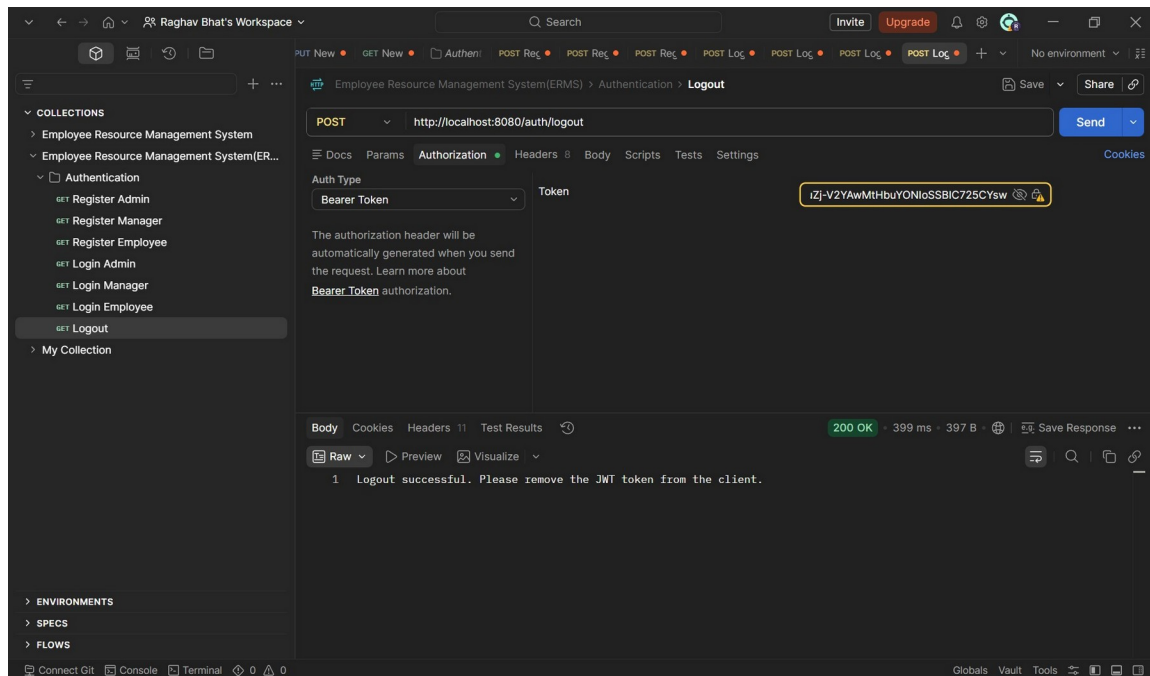


Figure 12: POST /auth/logout — session logout confirmation

6.2 Employee Management Module

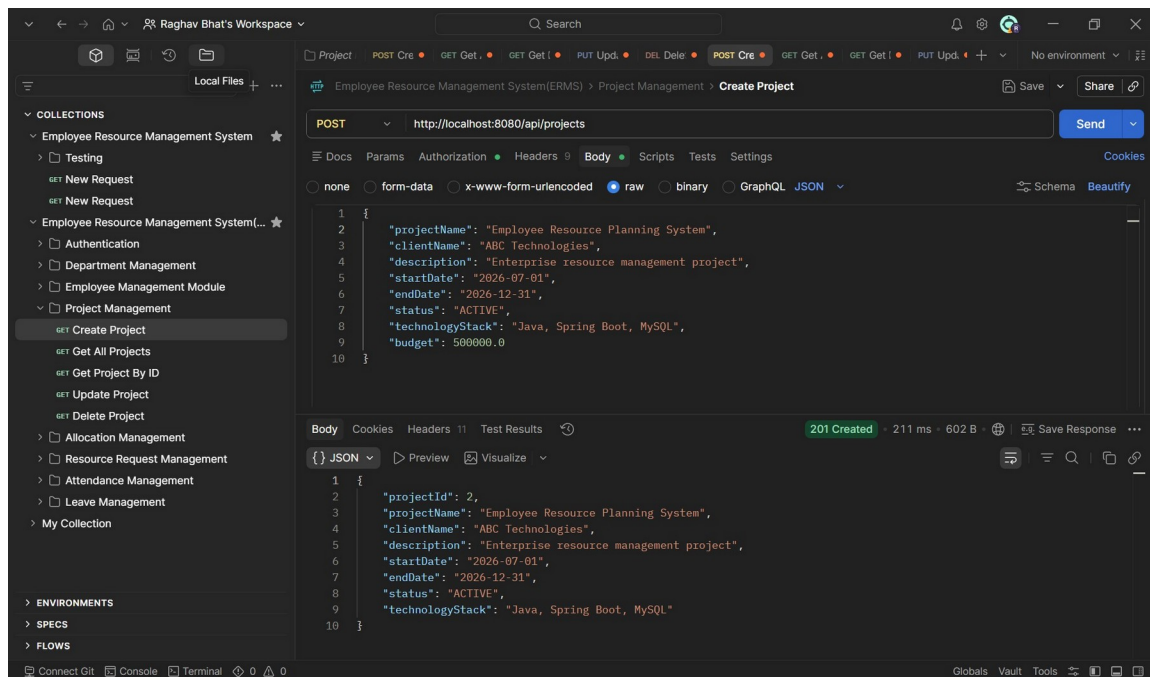


Figure 13: POST /api/employees — employee creation returning HTTP 201

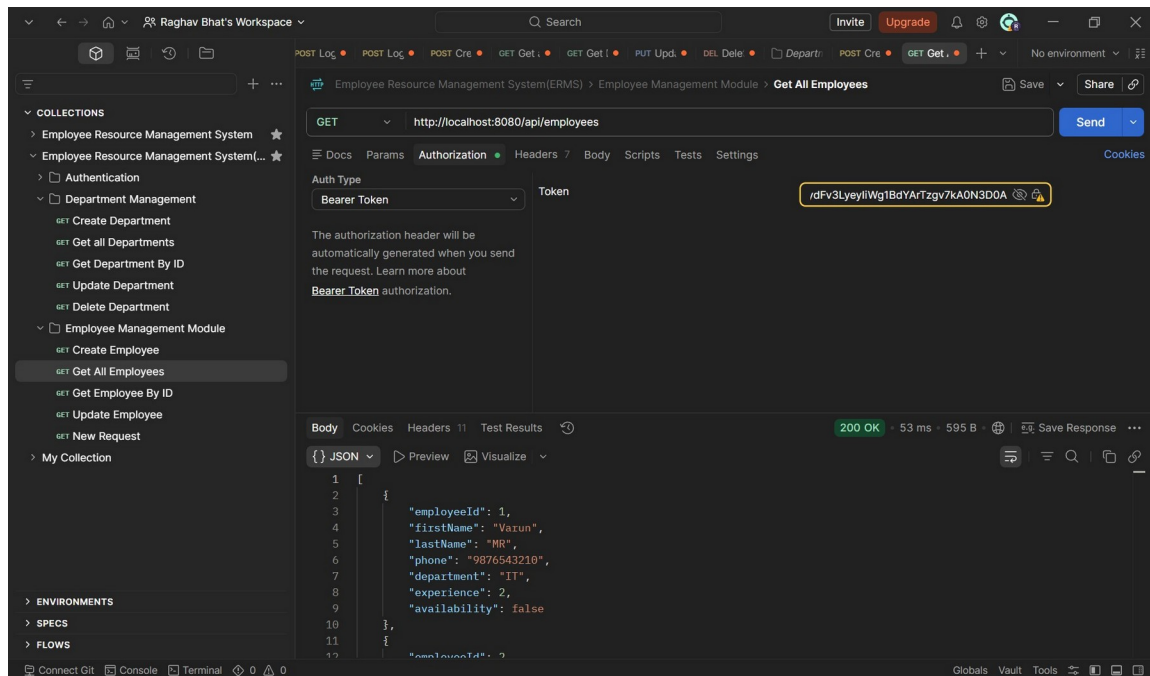


Figure 14: `GET /api/employees` — full employee listing (Bearer Token authorized)

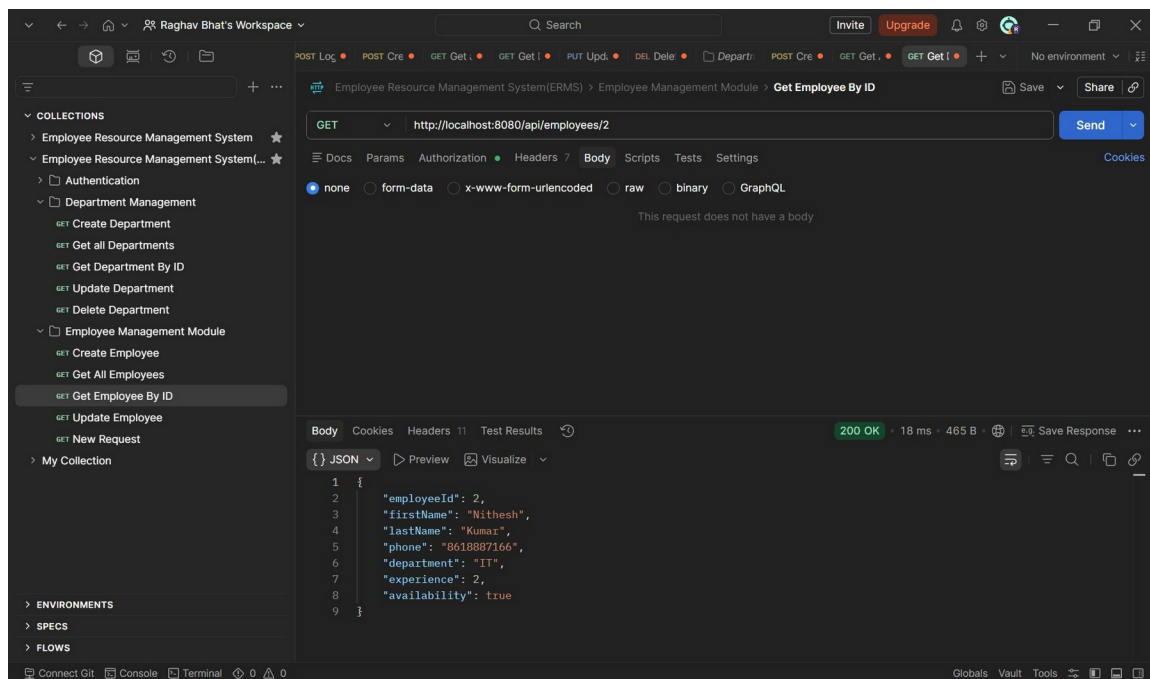


Figure 15: `GET /api/employees/{id}` — retrieve a single employee by ID

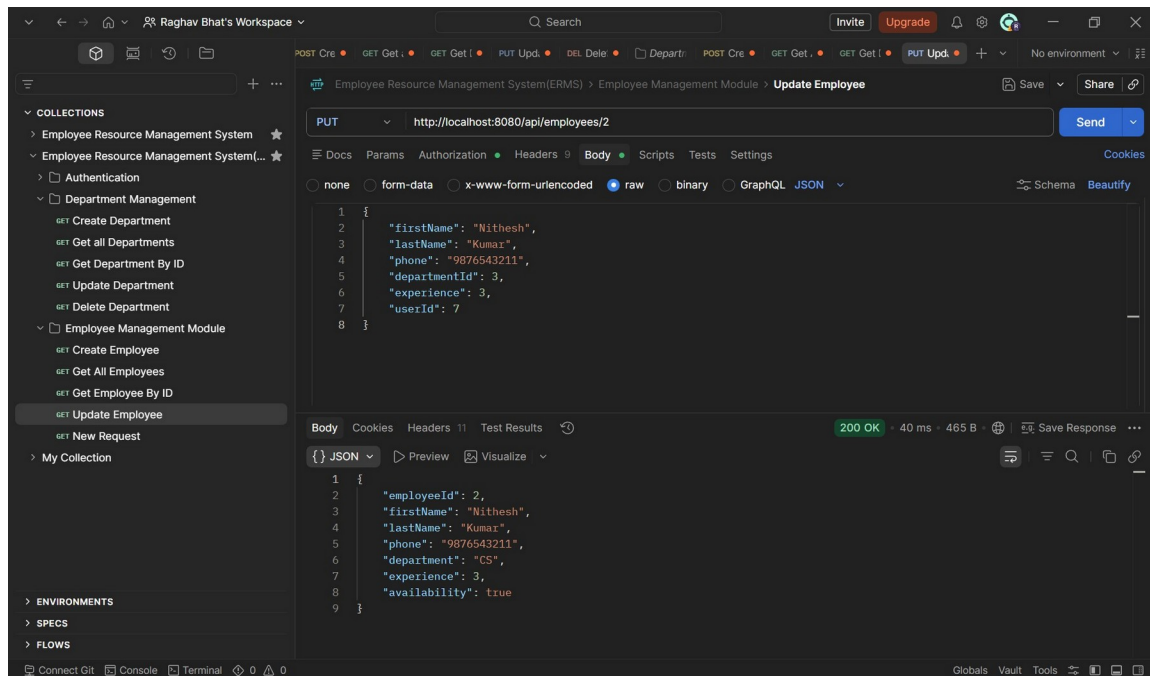


Figure 16: PUT /api/employees/{id} — update employee details

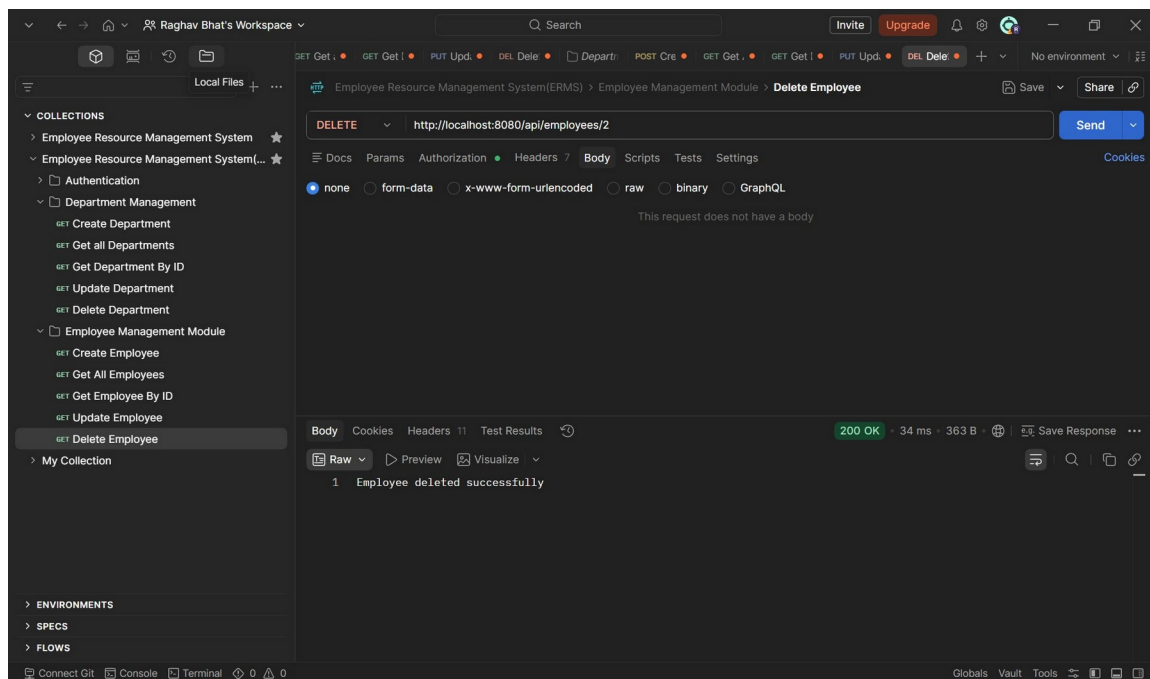


Figure 17: DELETE /api/employees/{id} — employee deletion confirmation

6.3 Project Management Module

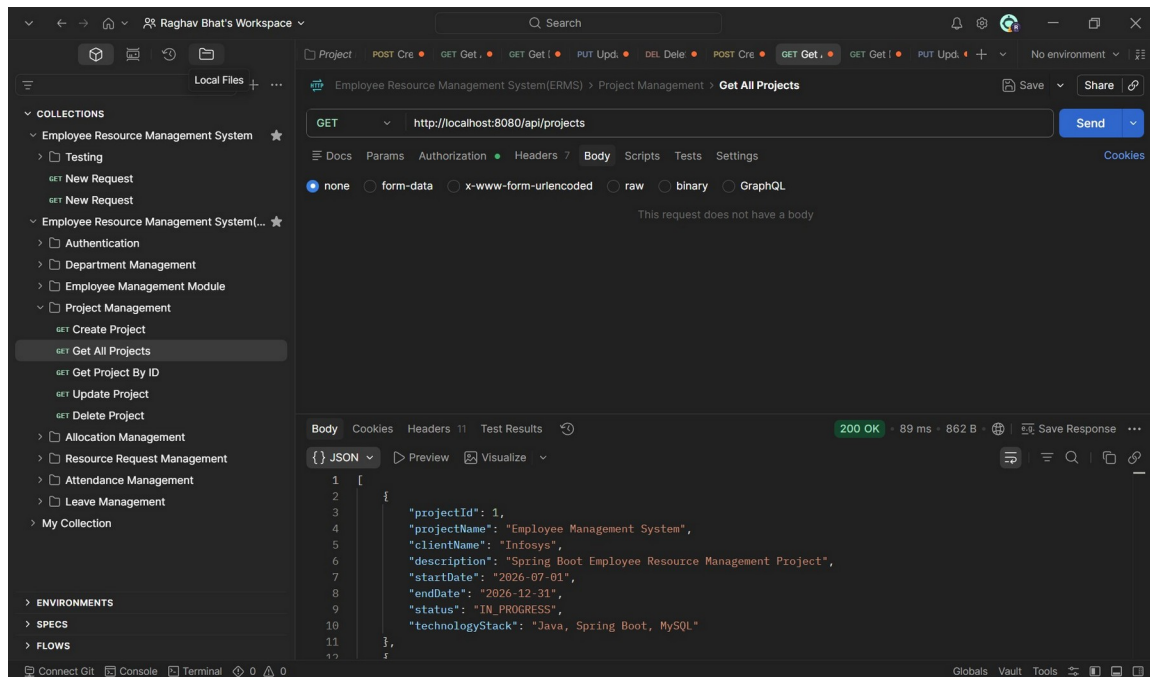


Figure 18: GET /api/projects — full project listing

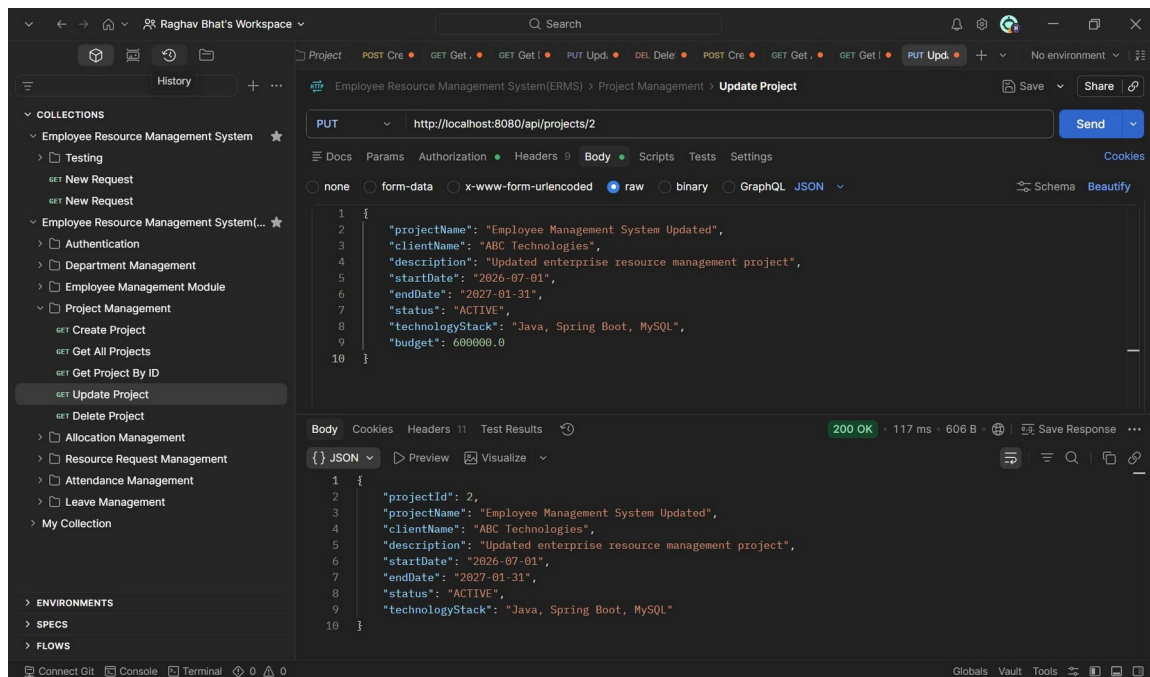


Figure 19: PUT /api/projects/{id} — update project details

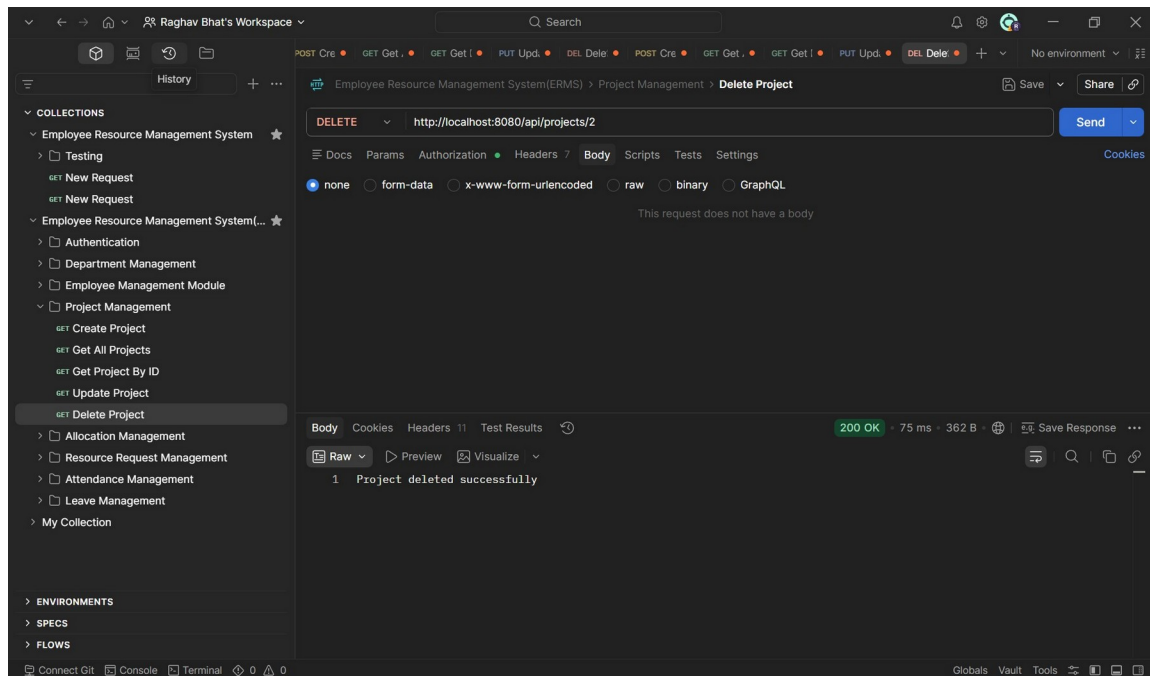


Figure 20: DELETE `/api/projects/{id}` — project deletion confirmation

6.4 Resource Allocation Module

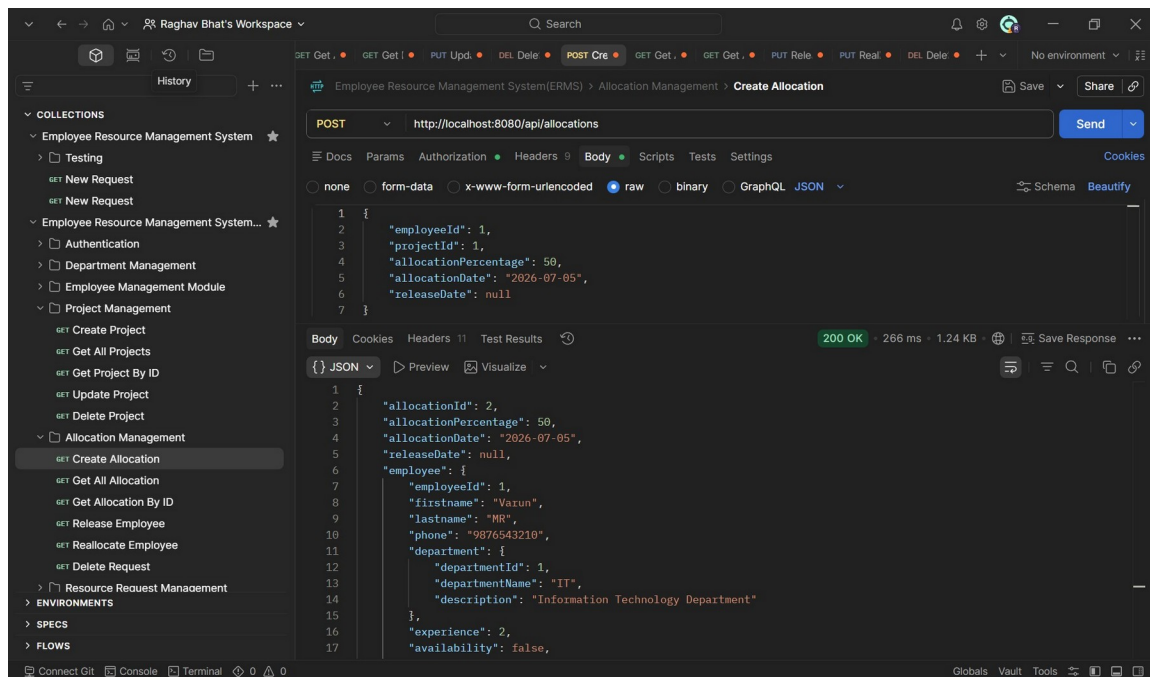


Figure 21: POST `/api/allocations` — create a new resource allocation

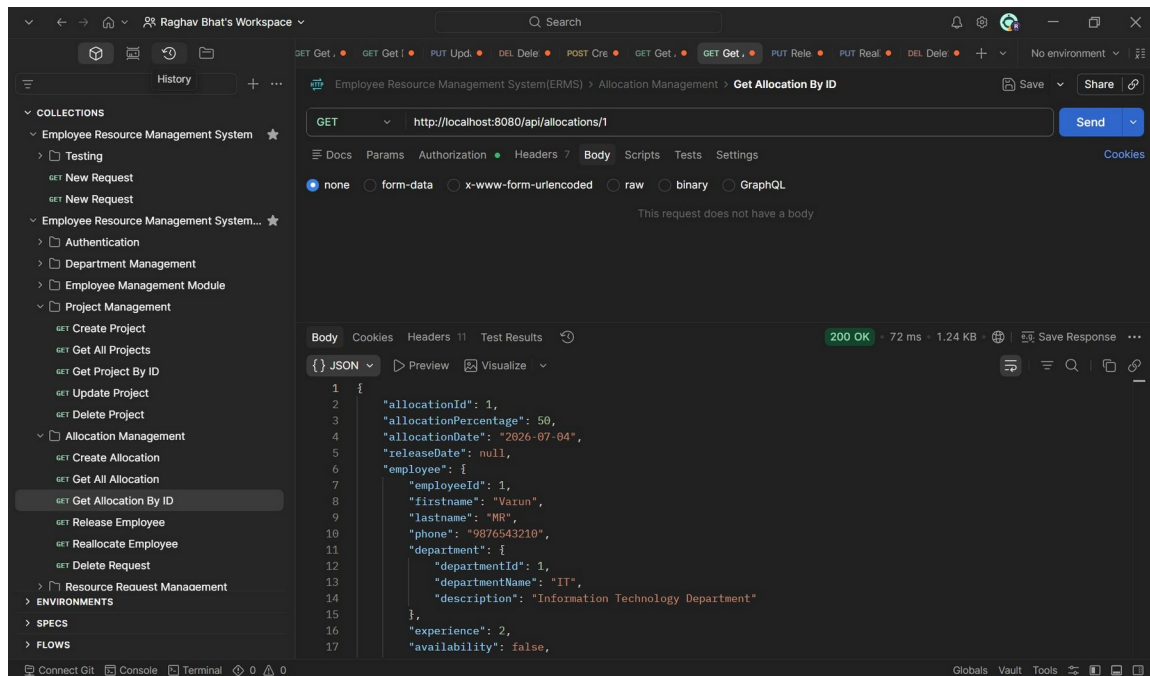


Figure 22: `GET /api/allocations/{id}` — retrieve allocation with nested employee/department detail

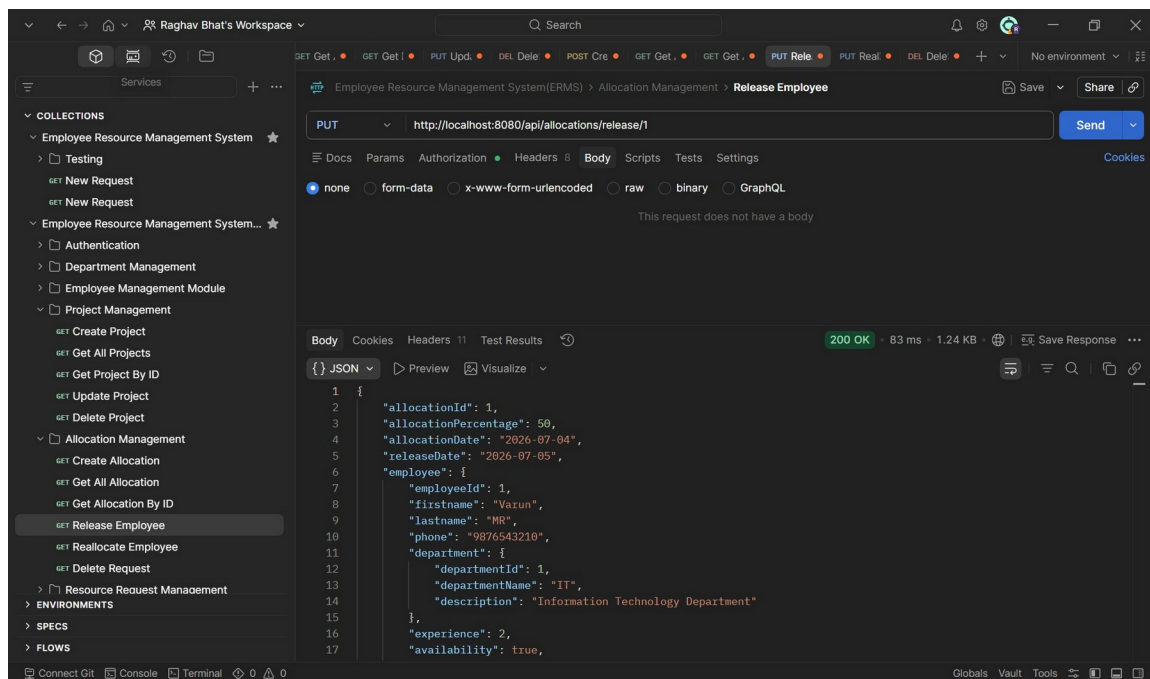


Figure 23: `PUT /api/allocations/release/{id}` — release employee from allocation

6.5 Representative Evidence (Summary Figures)

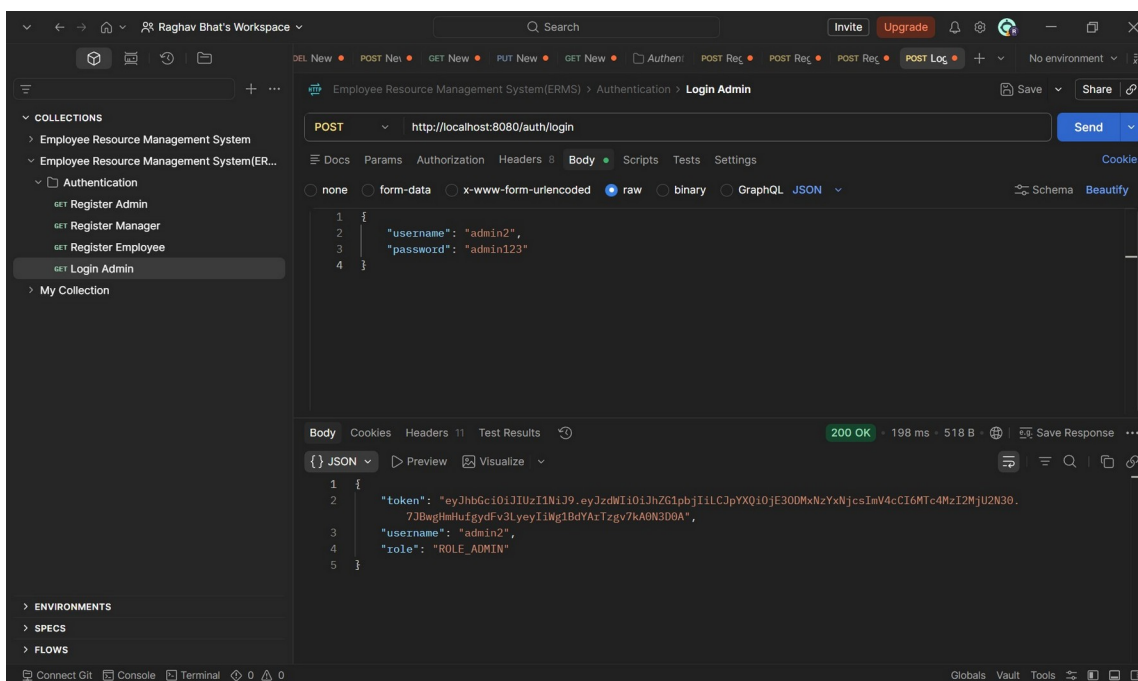


Figure 1: Admin login returning a signed JWT with role: `ROLE_ADMIN`

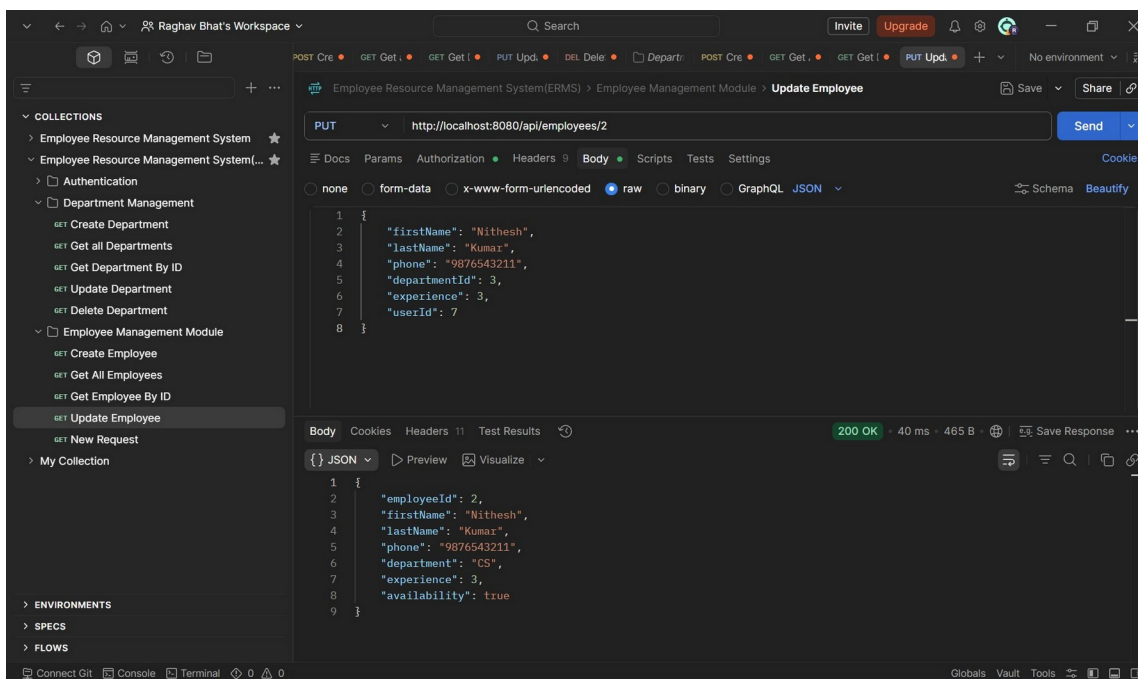


Figure 2: PUT /api/employees/2 — updating employee details

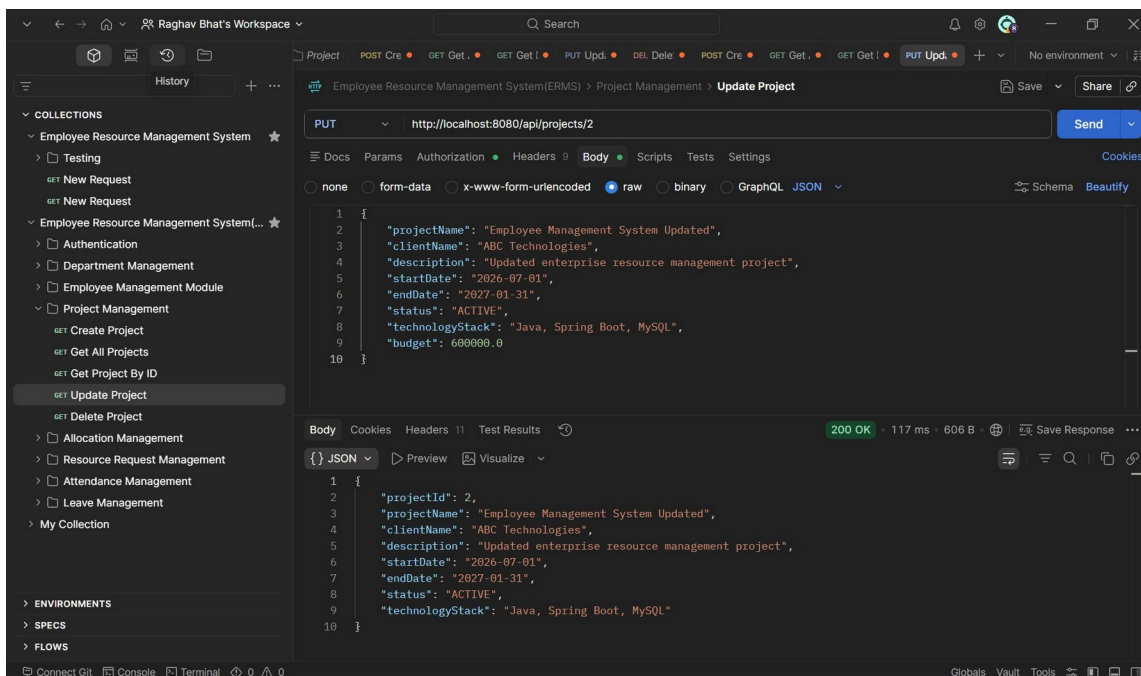


Figure 3: PUT /api/projects/2 — updating project details

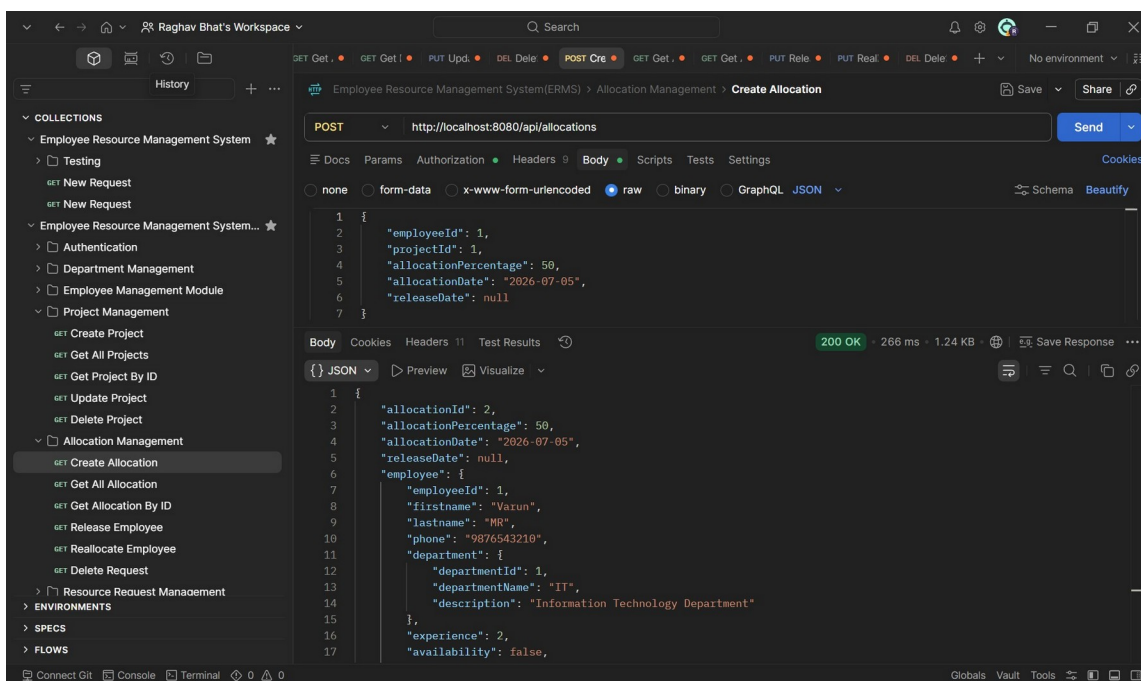


Figure 4: POST /api/allocations — creating a new allocation at 50%

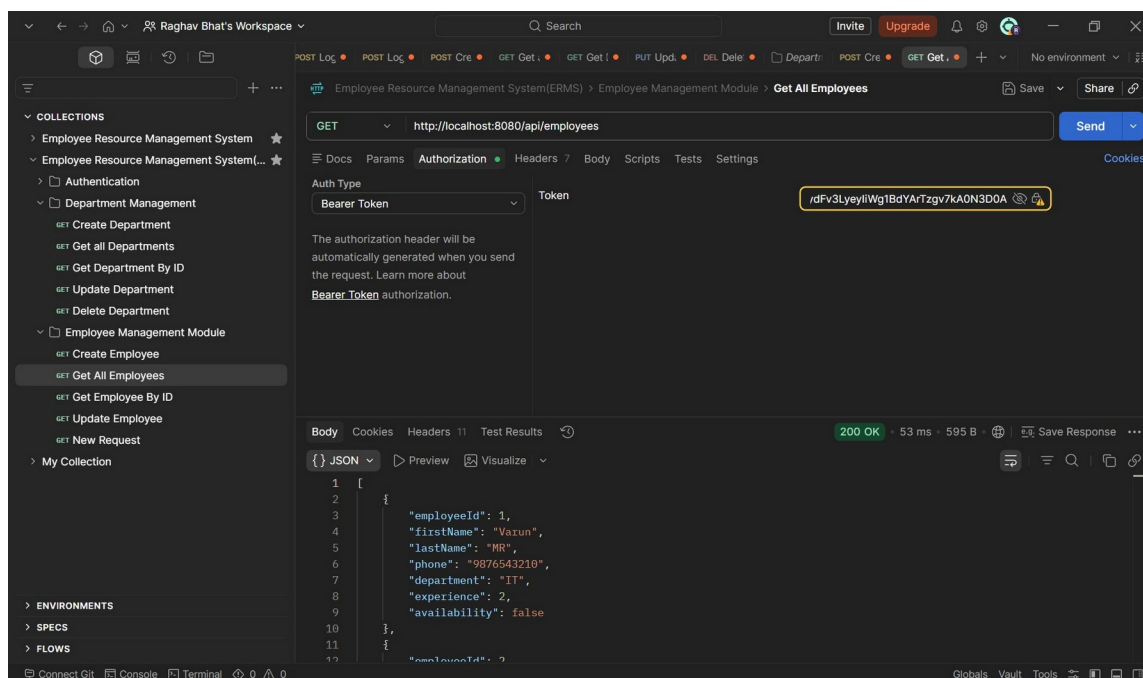


Figure 5: GET /api/employees — authorized via Bearer Token

7. Test Cases

Five representative test cases were executed against the ERASM application using Postman, covering authentication, core CRUD modules, resource allocation, and role-based security enforcement. All five passed, for a 100% pass rate.

Test Case 1: User Registration and Login Authentication using JWT

Test Case ID	TC-ERASM-001
Objective	To verify that a user can register with a specific role, log in with valid credentials to receive a signed JWT carrying the correct role claim, and log out to invalidate the client-side session.
Preconditions	The application server is running at <code>http://localhost:8080</code> and connected to MySQL; roleId values 1, 2, and 3 correspond to Admin, Manager, and Employee respectively.
Test Steps	1. Send POST <code>/auth/register</code> for an Admin, a Manager, and an Employee account, each with a unique username, email, password, and the corresponding roleId. 2. Send POST <code>/auth/login</code> for each account with valid credentials. 3. Inspect each login response for a signed JWT and confirm the role field matches the registered role. 4. Send POST <code>/auth/logout</code> using a valid Bearer token.
Expected Result	Each registration returns HTTP 200 with a confirmation message; each login returns HTTP 200 with a signed JWT and the correct role claim; logout returns HTTP 200 confirming the client should discard the token.
Actual Result	All three registrations returned HTTP 200 with "User Registered Successfully". All three logins returned HTTP 200 with a valid JWT and the correct role (ROLE_ADMIN, ROLE_MANAGER, ROLE_EMPLOYEE respectively). Logout returned HTTP 200 with the expected confirmation message.
Status	PASS

Test Case 2: Employee CRUD Operations

Test Case ID	TC-ERASM-002
Objective	To verify Create, Read, Update, and Delete operations on employee records exposed under <code>/api/employees</code> .
Preconditions	An authenticated JWT (Bearer token) is available; at least one department record exists to associate with the employee.
Test Steps	1. Send POST <code>/api/employees</code> with firstName, lastName, phone, departmentId, experience, and userId. 2. Send GET <code>/api/employees</code> to list all employees. 3. Send GET <code>/api/employees/{id}</code> to retrieve the created employee by ID. 4. Send PUT <code>/api/employees/{id}</code> to update phone and department. 5. Send DELETE <code>/api/employees/{id}</code> to remove the record.
Expected Result	Employee creation returns HTTP 201 with the generated employeeId; listing and retrieval return HTTP 200 with correct employee data; update returns HTTP 200 with modified fields persisted; deletion returns HTTP 200 with a confirmation message.

Actual Result	POST /api/employees returned HTTP 201 Created with a generated employeeId. GET /api/employees returned HTTP 200 with the full employee list. GET /api/employees/{id} returned HTTP 200 with correct data. PUT /api/employees/{id} returned HTTP 200 with the updated phone number and department. DELETE /api/employees/{id} returned HTTP 200 with “Employee deleted successfully”.
Status	PASS

Test Case 3: Project Management Operations

Test Case ID	TC-ERASM-003
Objective	To verify that project records exposed under /api/projects can be listed, updated, and deleted, with fields such as client name, technology stack, and budget correctly persisted.
Preconditions	An authenticated JWT is available; at least one project record already exists in the database.
Test Steps	1. Send GET /api/projects to list all projects and confirm existing fields. 2. Send PUT /api/projects/{id} to update the project name, description, dates, status, and budget. 3. Send GET /api/projects again to confirm the update persisted. 4. Send DELETE /api/projects/{id} to remove the project.
Expected Result	Listing returns HTTP 200 with an array of project objects; update returns HTTP 200 with all modified fields reflected; deletion returns HTTP 200 with a confirmation message and the project no longer appears on subsequent listing.
Actual Result	GET /api/projects returned HTTP 200 with the full list of projects. PUT /api/projects/2 returned HTTP 200 with the project renamed to “Employee Management System Updated”, status ACTIVE, and budget updated to 600000.0. DELETE /api/projects/2 returned HTTP 200 with “Project deleted successfully”.
Status	PASS

Test Case 4: Resource Allocation Lifecycle (Create, Retrieve, Release)

Test Case ID	TC-ERASM-004
Objective	To verify that an employee can be allocated to a project through /api/allocations, that the allocation can be retrieved with full employee and department detail, and that the employee can subsequently be released from the allocation.
Preconditions	An authenticated JWT is available; the target employee and project already exist and the employee is not already at 100% allocation.
Test Steps	1. Send POST /api/allocations with employeeId, projectId, allocationPercentage (50%), and allocationDate. 2. Send GET /api/allocations/{id} and confirm nested employee and department objects. 3. Send PUT /api/allocations/release/{id} to release the employee. 4. Re-fetch the allocation and confirm releaseDate is populated.

Expected Result	Allocation creation returns HTTP 200/201 with a generated allocationId and releaseDate initially null; retrieval returns HTTP 200 with full nested detail; release returns HTTP 200 with releaseDate populated and employee availability updated.
Actual Result	POST /api/allocations returned HTTP 200 with allocationId: 2, allocationPercentage: 50, and releaseDate: null. GET /api/allocations/1 returned HTTP 200 with complete nested detail. PUT /api/allocations/release/1 returned HTTP 200 with releaseDate populated (2026-07-05) and the employee's availability flag updated to true.
Status	PASS

Test Case 5: Role-Based Access Control using Spring Security

Test Case ID	TC-ERASM-005
Objective	To verify that JWT tokens issued for the three enforced roles (Admin, Manager, Employee) correctly carry their respective role claims, and that Bearer-token authorization is enforced on protected endpoints such as GET /api/employees.
Preconditions	Valid registered accounts exist for all three roles; a Bearer token is available for each.
Test Steps	1. Log in as Admin, Manager, and Employee and inspect each JWT's decoded role claim. 2. Call GET /api/employees with no Authorization header and confirm it is rejected. 3. Call the same endpoint with a valid Bearer token and confirm access is granted. 4. Repeat with each role token to confirm all authenticated roles can reach this endpoint, while write-restricted endpoints remain governed by @PreAuthorize.
Expected Result	Requests without a valid Bearer token are rejected; requests with a valid token and an authorized role return HTTP 200; each JWT's role claim exactly matches the account's registered role.
Actual Result	Login responses correctly returned role: ROLE_ADMIN, ROLE_MANAGER, and ROLE_EMPLOYEE for the respective accounts. GET /api/employees was successfully authorized using a Bearer token, returning HTTP 200 with the full employee list; the endpoint is inaccessible without a valid token.
Status	PASS

Overall Test Execution Summary

Total Test Cases Executed	5
Test Cases Passed	5
Test Cases Failed	0
Pass Percentage	100%